



PyQuest

Information- & Aufgabenblatt

Kapitel 3: Datentypen

Zahlen, Text und Wahrheitswerte – und wie man Typen ineinander umwandelt.

Die vier Datentypen

Jeder Wert in Python hat einen **Datentyp** – er sagt, um welche Art von Information es sich handelt. Für den Anfang sind vier Typen wichtig:

- **int** – Ganze Zahlen: 123, -5, 0
- **float** – Kommazahlen: 1.23, 3.14, -0.5
- **str** – Zeichenketten (Text): "Hello", "Ada"
- **bool** – Wahrheitswerte: True oder False

Warum ist der Typ wichtig? Weil Python je nach Typ **anders** rechnet. $3 + 4$ ergibt 7 (Zahlen), aber $"3" + "4"$ ergibt "34" (Text wird aneinandergehängt).

Mit der Funktion **type(wert)** findest du heraus, welchen Typ ein Wert hat:

```
print(type(123))    → <class 'int'>
print(type(1.5))    → <class 'float'>
print(type("hi"))   → <class 'str'>
print(type(True))   → <class 'bool'>
```

Aufgabe 1: Welchen Datentyp hat der Wert 3.14?

- ☐ a) int
- ☐ b) float
- ☐ c) str

**Aufgabe 2: Und welchen Datentyp hat True?**

- ☐ a) str
- ☐ b) int
- ☐ c) bool

In der Praxis wählt man für jede Information den passenden Typ. Ein Fitnessstudio speichert z. B.:

- Mitglieds-ID → **int** (101)
- Vorname → **str** ("Max")
- Premium-Mitglied? → **bool** (True)
- Durchschnittsbewertung → **float** (4.5)

Aufgabe 3: Finde den Typ heraus: Gib mit `type()` und `print()` den Datentyp des Werts 4.5 aus.

Dein Code:

Zahlen (int und float)

Python kennt zwei Arten von Zahlen:

- **int** (Ganzzahlen): 5, -3, 100
- **float** (Kommazahlen): 3.14, -0.5, 2.0

Im Deutschen schreibt man ein Komma (3,14) – in Python **immer einen Punkt**: 3.14.

Die Grundrechenarten funktionieren wie erwartet. Klick auf **Ausführen**:

Beispiel

```
print(3 + 4)
print(10 - 2)
print(3 * 5)
print(10 / 4)
```



Bei der Division gibt es eine Besonderheit:

- / teilt normal und liefert **immer** eine Kommazahl: $10 / 4 \rightarrow 2.5$, sogar $8 / 2 \rightarrow 4.0$
- // (Ganzzahldivision) liefert nur den **ganzen** Teil ohne Rest: $10 // 4 \rightarrow 2$
- % (Modulo) liefert den **Rest** einer Division: $10 \% 4 \rightarrow 2$

Aufgabe 4: Was gibt `print(7 // 2)` aus?

- ☐ a) 3.5
- ☐ b) 3
- ☐ c) 4

Aufgabe 5: Und was gibt `print(7 % 2)` aus?

- ☐ a) 1
- ☐ b) 3
- ☐ c) 3.5

Mit **kannst du** potenzieren (**hoch rechnen**): `23` bedeutet $2 \cdot 2 \cdot 2 = 8$. Das brauchst du z. B. für Flächen (seite **2**) oder Volumen (seite **3**).

Aufgabe 6: Berechne 2 hoch 10 (`210`) und gib das Ergebnis aus (1024).

Dein Code:



Aufgabe 7: Ein Trainingsraum ist 20 m lang, 15 m breit und 3 m hoch. Berechne das Volumen (Länge · Breite · Höhe) und gib es aus (900).

```
laenge = 20  
breite = 15  
hoehe = 3
```

Dein Code:

Text (Strings)

Text wird in Python **str** (String) genannt und steht in **Anführungszeichen**: "Hallo" oder 'Hallo' (beide gehen, Hauptsache gleich am Anfang und Ende).

Mit **+** kannst du Texte **aneinanderhängen** (verketteten).

Zwei Texte und ein Leerzeichen dazwischen verketteten:

Beispiel

```
vorname = "Ada"  
nachname = "Lovelace"  
print(vorname + " " + nachname)
```

Achtung, häufiger Anfängerfehler: Du kannst Text **nicht** direkt mit einer Zahl verketteten.

```
print("Alter: " + 16) → TypeError!
```

Text (str) und Zahl (int) sind verschiedene Typen. Du musst die Zahl erst mit **str()** in Text umwandeln:

```
print("Alter: " + str(16)) → Alter: 16
```

**Aufgabe 8: Warum wirft `print("Punkte: " + 5)` einen Fehler?**

- ☐ a) Weil man Text und Zahl nicht direkt verbinden kann
- ☐ b) Weil "Punkte: " zu lang ist
- ☐ c) Weil 5 keine gültige Zahl ist

Aufgabe 9: Wandle die Zahl `alter` in Text um, damit die Verkettung funktioniert.

```
print("Alter: " + ____(alter))
```

Dein Code:

Ein bequemer Trick: `print()` kann auch **mehrere Werte** ausgeben, getrennt durch **Kommas**. Dann brauchst du kein `str()` und Python setzt automatisch ein Leerzeichen dazwischen:

```
alter = 16  
print("Alter:", alter)    → Alter: 16
```

Aufgabe 10: Lege `stadt = "Weingarten"` an und gib Ich wohne in Weingarten aus (nutze Verkettung mit `+`).

Dein Code:



Wahrheitswerte (bool)

Ein **Wahrheitswert** (bool) kennt nur zwei Zustände: **True** (wahr) oder **False** (falsch).

Wichtig: True und False schreibt man **groß** und **ohne** Anführungszeichen – sie sind keine Texte, sondern eigene Werte.

Wahrheitswerte entstehen oft durch **Vergleiche**. Das Ergebnis eines Vergleichs ist immer True oder False:

- == gleich, != ungleich
- < kleiner, > größer
- <= kleiner-gleich, >= größer-gleich

Vergleiche ergeben bool-Werte:

Beispiel

```
print(5 > 3)
print(5 == 3)
print(type(5 > 3))
```

Aufgabe 11: Was gibt print(10 >= 10) aus?

- ☐ a) True
- ☐ b) False
- ☐ c) Fehler

Aufgepasst beim Vergleichen: == (zwei Gleichheitszeichen) **vergleicht**, = (ein Gleichheitszeichen) **weist zu**. alter = 18 speichert 18 in alter. alter == 18 prüft, ob alter gleich 18 ist. Das ist der häufigste Anfängerfehler!

Aufgabe 12: Prüfe mit einem Vergleich, ob 7 kleiner ist als 3, und gib das Ergebnis aus (soll False sein).

Dein Code:



Typen umwandeln (explizit)

Oft musst du einen Wert von einem Typ in einen anderen **umwandeln** (konvertieren). Wenn **du** das bewusst machst, heißt es **explizite** Typkonvertierung. Dafür gibt es eingebaute Funktionen:

- **int(wert)** → macht eine Ganzzahl daraus
- **float(wert)** → macht eine Kommazahl daraus
- **str(wert)** → macht Text daraus

Ein float in einen int umwandeln – dabei geht die Nachkommastelle verloren (es wird **abgeschnitten**, nicht gerundet):

Beispiel

```
float_wert = 12.7
int_wert = int(float_wert)
print(int_wert)
```

Aufgabe 13: Was gibt print(int(3.9)) aus?

- ☐ a) 4
- ☐ b) 3
- ☐ c) 3.9

Besonders wichtig: Ein **Text, der wie eine Zahl aussieht**, ist trotzdem Text. Erst int() oder float() macht daraus eine echte Zahl, mit der du rechnen kannst:

```
zahl_text = "123"
zahl = int(zahl_text)
print(zahl + 10) → 133
```

Aufgabe 14: Der Text "4.23" soll in eine echte Kommazahl umgewandelt werden. Wandle ihn mit float() um, speichere ihn in zahl und gib zahl aus.

```
text = "4.23"
```

Dein Code:



Vorsicht: `int()` klappt nur, wenn der Text wirklich eine Zahl ist. `int("5")` ergibt 5, aber `int("hallo")` wirft einen **ValueError** – „ungültiger Wert“. Das passiert oft, wenn eine Nutzereingabe keine Zahl enthält.

Auch bool-Werte kann man umwandeln. Bei `int(True)` kommt 1 heraus, bei `int(False)` kommt 0 heraus. So kann man mit Wahrheitswerten sogar rechnen (z. B. zählen, wie oft etwas True war).

Aufgabe 15: Wandle den bool-Wert True mit `int()` in eine Zahl um, speichere sie in `zahl` und gib sie aus (soll 1 sein).

Dein Code:

Typen umwandeln (implizit)

Manchmal wandelt **Python selbst** einen Typ um, ganz automatisch – ohne dass du eine Funktion aufrufst. Das nennt man **implizite** Typkonvertierung.

Das passiert, wenn in einer Rechnung verschiedene Zahltypen zusammenkommen. Python macht dann aus dem „kleineren“ Typ den „größeren“, damit die Rechnung aufgeht.

Hier wird ein `int` mit einem `float` addiert. Python wandelt die 10 automatisch in 10.0 um – das Ergebnis ist ein `float`:

Beispiel

```
int_wert = 10
float_wert = 20.5
ergebnis = int_wert + float_wert
print(ergebnis)
print(type(ergebnis))
```

Aufgabe 16: Welchen Typ hat das Ergebnis von `5 + 2.0`?

- ☐ a) `int`
- ☐ b) `float`
- ☐ c) `str`



Der Unterschied zusammengefasst:

- **Explizit:** **Du** wandelst um, mit `int()`, `float()`, `str()`. (Du triffst die Entscheidung.)
- **Implizit:** **Python** wandelt automatisch um, z. B. bei `int + float`. (Python entscheidet je nach Kontext.)

Text (`str`) wandelt Python übrigens **nie** automatisch um – `"3" + 4` bleibt ein Fehler. Da musst **du** ran.

Aufgabe 17: Was ist bei `x = y + z` (y ist int, z ist float) der Fall?

- ☐ a) Explizite Konvertierung – du rufst eine Funktion auf
- ☐ b) Implizite Konvertierung – Python macht y automatisch zum float
- ☐ c) Gar keine Konvertierung

Aufgabe 18: Addiere die Ganzzahl 7 und die Kommazahl 2.5, speichere das Ergebnis in `summe` und gib es aus. Python kümmert sich automatisch um die Umwandlung (Ergebnis: 9.5).

Dein Code: